

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №2
«Модульное тестирование»

Выполнил:
Студент группы ИУ5-31Б
Блинкова Ксения

Проверил:
Гапанюк Ю. Е.

2025 г.

Листинг программы

refactored_rk1.py

```
from operator import itemgetter

class Driver:
    def __init__(self, id, fio, salary, park_id):
        self.id = id
        self.fio = fio
        self.salary = salary
        self.park_id = park_id

class Park:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class DriverPark:
    def __init__(self, park_id, driver_id):
        self.park_id = park_id
        self.driver_id = driver_id

class DataService:
    def __init__(self):
        self.parks = [
            Park(1, 'Восточный автопарк'),
            Park(2, 'Западный автопарк'),
            Park(3, 'Северный автопарк'),
            Park(11, 'Восточный отдел логистики'),
            Park(22, 'Западный отдел транспорта'),
            Park(33, 'Центральный отдел грузоперевозок'),
        ]

        self.drivers = [
            Driver(1, 'Алексеев', 25000, 1),
            Driver(2, 'Антонов', 35000, 2),
            Driver(3, 'Кузнецов', 45000, 3),
            Driver(4, 'Иванов', 35000, 3),
            Driver(5, 'Петров', 25000, 3),
            Driver(6, 'Сидоров', 40000, 11),
            Driver(7, 'Никитин', 30000, 22),
        ]

        self.drivers_parks = [
            DriverPark(1, 1),
            DriverPark(2, 2),
            DriverPark(3, 3),
            DriverPark(3, 4),
            DriverPark(3, 5),
            DriverPark(11, 6),
            DriverPark(22, 7),
```

```
        DriverPark(33, 1),
        DriverPark(33, 3),
    ]
```

```
class DriverParkService:
```

```
    def __init__(self, data_service):
        self.data_service = data_service
```

```
    def get_one_to_many(self):
        return [
            (d.fio, d.salary, p.name)
            for p in self.data_service.parks
            for d in self.data_service.drivers
            if d.park_id == p.id
        ]
```

```
    def get_many_to_many(self):
        many_to_many_temp = [
            (p.name, dp.park_id, dp.driver_id)
            for p in self.data_service.parks
            for dp in self.data_service.drivers_parks
            if p.id == dp.park_id
        ]
```

```
        return [
            (d.fio, d.salary, park_name)
            for park_name, park_id, driver_id in many_to_many_temp
            for d in self.data_service.drivers
            if d.id == driver_id
        ]
```

```
    def task_a1(self):
        one_to_many = self.get_one_to_many()
        return sorted(one_to_many, key=itemgetter(2, 0))
```

```
    def task_a2(self):
        one_to_many = self.get_one_to_many()
        parks_total_salary = []

        for p in self.data_service.parks:
            p_drivers = [item for item in one_to_many if item[2] == p.name]

            if p_drivers:
                total_salary = sum(salary for _, salary, _ in p_drivers)
                parks_total_salary.append((p.name, total_salary))

        return sorted(parks_total_salary, key=itemgetter(1), reverse=True)
```

```
    def task_a3(self):
        many_to_many = self.get_many_to_many()
        result = {}

        for p in self.data_service.parks:
```

```

        if 'отдел' in p.name.lower():
            # Находим всех водителей в этом автопарке
            p_drivers = [item for item in many_to_many if item[2] == p.name]
            # Извлекаем только фамилии водителей
            drivers_names = [driver_name for driver_name, _, _ in p_drivers]
            result[p.name] = drivers_names

    return result

def print_results(service):

    print('Задание A1')
    print('Список всех связанных водителей и автопарков, отсортированный по автопаркам:')

    for driver_fio, salary, park_name in service.task_a1():
        print(f" {park_name}: {driver_fio} - {salary} руб.")

    print('\nЗадание A2')
    print('Список автопарков с суммарной зарплатой водителей, отсортированный по суммарной зарплате:')

    for park_name, total_salary in service.task_a2():
        print(f" {park_name}: {total_salary} руб.")

    print('\nЗадание A3')
    print('Список автопарков, у которых в названии присутствует слово "отдел", и список работающих в них водителей:')

    for park_name, drivers_list in service.task_a3().items():
        drivers_str = ', '.join(drivers_list)
        print(f" {park_name}: {drivers_str}")

def main():
    data_service = DataService()
    driver_park_service = DriverParkService(data_service)

    print_results(driver_park_service)

if __name__ == '__main__':
    main()

```

test_rk2.py

```
import pytest
from refactored_rk1 import DataService, DriverParkService

class TestDriverParkService:

    @pytest.fixture
    def service(self):
        return DriverParkService(DataService())

    def test_task_a1(self, service):
        result = service.task_a1()

        assert len(result) == 7

        park_names = [park_name for _, _, park_name in result]
        assert park_names == sorted(park_names)

        assert result[0][2] == 'Восточный автопарк'
        assert result[0][0] == 'Алексеев'
        assert result[0][1] == 25000

        northern_park_items = [item for item in result if item[2] == 'Северный автопарк']
        northern_drivers = [driver for driver, _, _ in northern_park_items]
        assert northern_drivers == ['Иванов', 'Кузнецов', 'Петров']

    def test_task_a2(self, service):
        result = service.task_a2()

        assert len(result) == 5

        salaries = [salary for _, salary in result]
        assert salaries == sorted(salaries, reverse=True)

        result_dict = dict(result)
        assert result_dict['Северный автопарк'] == 105000
        assert result_dict['Восточный отдел логистики'] == 40000
        assert result_dict['Западный автопарк'] == 35000
        assert result_dict['Западный отдел транспорта'] == 30000
        assert result_dict['Восточный автопарк'] == 25000

    def test_task_a3(self, service):
        result = service.task_a3()

        assert isinstance(result, dict)
        assert len(result) == 3

        assert 'Восточный отдел логистики' in result
        assert 'Западный отдел транспорта' in result
        assert 'Центральный отдел грузоперевозок' in result

        assert result['Восточный отдел логистики'] == ['Сидоров']
        assert result['Западный отдел транспорта'] == ['Никитин']
```

```
central_drivers = result['Центральный отдел грузоперевозок']
assert len(central_drivers) == 2
assert 'Алексеев' in central_drivers
assert 'Кузнецов' in central_drivers

if __name__ == '__main__':
    pytest.main([__file__, '-v'])
```

Результат выполнения:

```
(venv) ksu@LAPTOP-04SGK9AC:~/Blinkova_Labs_2025$ python -m pytest test_rk2.py -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /home/ksu/Blinkova_Labs_2025/venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ksu/Blinkova_Labs_2025
collected 3 items

test_rk2.py::TestDriverParkService::test_task_01 PASSED
test_rk2.py::TestDriverParkService::test_task_02 PASSED
test_rk2.py::TestDriverParkService::test_task_03 PASSED

===== 3 passed in 0.02s =====
```

